DSA03-NATURAL LANGUAGE PROCESSING
ASSIGNMENT-3

ASSIGNMENT 3 – CO3

CO3 Coverage: Context-Free Grammars for English syntax; Context-free rules and trees; Sentence-level constructions; Agreement; Subcategorization; Parsing (Top down, Earley); Feature structures; Probabilistic Context-Free Grammars

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 06/11/2026

Assignment Title: Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Syntactic Analysis for a University Helpdesk Assistant A university is building a text-based helpdesk assistant that answers student and staff queries about timetables, examinations, hostels, fees, library services and placements. Before any meaning can be extracted, every query must be assigned a reliable syntactic structure. The NLP team has decided to build a grammar-based parser for a controlled but realistic fragment of English, to resolve ambiguity with a probabilistic model, and to compare parsing algorithms on accuracy and cost.

You are the language-engineering team. You must investigate how English syntax can be captured by context-free rules, how agreement and subcategorization constrain those rules, how parsing algorithms search for trees, and how probabilities select the best tree when a query is ambiguous.

Planning data and constraints

- Grammar fragment: at least 40 phrase-structure rules and a lexicon of at least 60 words covering the helpdesk domain.
- Sample query set (build and document your own): at least 30 sentences, including declaratives, imperatives, yes/no questions, wh-questions and at

least 5 structurally ambiguous sentences (e.g., “Show the students with the late fee notice”, “Book the room near the library for the seminar”).

• Required phenomena: subject–verb agreement, determiner–noun number agreement, pronoun case, and at least five verb subcategorization frames. • The grammar must be written in Chomsky-style CFG notation; for the Earley and probabilistic tasks the grammar may be converted to a binarised or CNF equivalent form where needed, and the conversion must be shown. • Treebank probabilities: a small hand-annotated corpus of at least 20 parsed sentences may be used to estimate rule probabilities (maximum-likelihood estimates with smoothing explained).

• Any manual effort (grammar writing, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The helpdesk is to be extended to accept noisy, conversational and multi-clause student messages (e.g., “Can you tell me when the exams that were postponed will be held?”). Re-evaluate your original grammar and parser and decide whether to retain, modify or redesign them.

TASK 1 – Design a Context-Free Grammar for English Syntax 5 Marks

Guiding Enquiry

How can the core structure of English be captured by a finite set of context-free rules, and what design decisions make the grammar both expressive and manageable?

Investigate and interpret:

• Define the formal CFG **G = (N, Σ, R, S)** for your helpdesk fragment and state clearly the non-terminals, terminals (lexicon), rules and start symbol. • Write rules for the major constituents: sentence (S), noun phrase (NP), verb phrase (VP), prepositional phrase (PP), adjective phrase, and coordination. Include recursive rules (e.g., NP → NP PP) and explain how they permit unbounded structure.

• Separate the grammar proper from the lexicon; list the parts of speech used and justify the granularity you selected.

• Explain the notions of constituency, head, and phrasal category with tests (substitution, movement, coordination) applied to at least three of your sentences.

• Show that your grammar is context-free by construction and discuss one English phenomenon that strains the context-free assumption.

Evidence Expected

A complete, well-organised CFG with lexicon, justified categories and worked constituency tests.

TASK 2 – Context-Free Rules, Derivations and Parse Trees 3 Marks

Guiding Enquiry

How are sentences generated from the grammar, and how are trees used to represent and compare syntactic analyses?

Investigate and interpret:

- For five sentences from your query set, give a leftmost derivation and a rightmost derivation, and draw the corresponding parse tree.
- Represent each tree in both bracketed notation and tree diagram form, and compute its height, number of nodes and number of rule applications. • Demonstrate ambiguity: for at least two sentences, give two distinct parse trees and describe the difference (e.g., PP-attachment, coordination scope). • Explain the relationship between derivations and trees, and why many derivations may correspond to one tree.
- State a procedure to test whether a given string belongs to L(G) and discuss its limits.

Evidence Expected

Correct derivations, accurately drawn trees, and a clear analysis of structural ambiguity.

TASK 3 – Sentence-Level Constructions 4 Marks

Guiding Enquiry

Which sentence types must the helpdesk understand, and how does the grammar represent each of them?

Investigate and interpret:

- Provide rules for the four main sentence-level constructions: declaratives, imperatives, yes/no questions and wh-questions (subject and non-subject). • Handle auxiliary inversion, wh-movement (represented through a gap or an additional rule), and subordinate and relative clauses (e.g., "the exams that were postponed").
- Show parse trees for at least one sentence of each construction type and note the structural differences from the declarative.
- Discuss how coordination ("fees and hostel charges") and long-distance dependencies are

treated and where the plain CFG becomes unwieldy. • Analyse the effect of each new construction on grammar size and ambiguity.

Evidence Expected

Rules and trees for each sentence type, with an analysis of how the added constructions affect the grammar.

TASK 4 – Agreement 3 Marks

Guiding Enquiry

How can a grammar prevent sentences such as "*The students submits the form" without producing an enormous set of rules?

Investigate and interpret:

- Identify the agreement phenomena in your fragment: subject–verb (number, person), determiner–noun, and pronoun case.
- First, enforce agreement by multiplying non-terminals (e.g., S → NP-sg VP-sg | NP-pl VP-pl). Count how many rules this requires for your grammar and explain the growth.
- Show how the rule explosion worsens as person, number, case and gender are combined, and compute the number of rules needed in the worst case. • Propose a more economical alternative (feature-based rules) and show informally how it prevents ungrammatical strings; the formal treatment follows in Task 8.
- Test the original and agreement-aware grammars on at least six grammatical and six ungrammatical sentences and tabulate the results.

Evidence Expected

A quantified comparison of the baseline and agreement-aware grammars with test results.

TASK 5 – Subcategorization 4 Marks

Guiding Enquiry

How does the grammar know which complements a verb may take, and how can this knowledge be encoded?

Investigate and interpret:

- Define subcategorization (valence) and list at least five frames for helpdesk verbs, e.g., intransitive ("The portal crashed"), transitive ("Submit the form"), ditransitive ("Send me the receipt"), sentential complement ("Tell me that the fee is paid"), and PP complement ("Apply for a hostel room").

- Encode the frames first with distinct verb categories (V-intr, V-trans, V-ditrans, V-comp, …) and then explain the drawbacks of this approach.
- Distinguish complements from adjuncts using tests, with examples.
- Demonstrate how the grammar rejects over- and under-complemented sentences ("*Submit", "*The portal crashed the form") and show this in the parse results.
- Discuss verbs with multiple frames and how this changes ambiguity and parsing cost.

Evidence Expected

A subcategorization lexicon, grammar rules enforcing it, and tested accept/reject examples.

TASK 6 – Top-Down Parsing 4 Marks

Guiding Enquiry

How does a top-down parser search for a tree rooted in S, and what are the strengths and weaknesses of this strategy?

Investigate and interpret:

- Describe the top-down (recursive-descent, goal-driven) algorithm in pseudocode, including how it expands non-terminals, matches words and backtracks.
- Trace the parser step by step on one unambiguous and one ambiguous sentence, showing the stack/agenda, the rule chosen and each backtracking point.
- Explain the problems of left recursion (e.g., NP → NP PP) and repeated re parsing of the same subtrees, and show these problems on a concrete example.
- Show how left-recursion elimination or a bottom-up filter (left-corner) addresses these issues and discuss the effect on the resulting trees.
- Analyse the worst-case time behaviour and the number of redundant sub parses for a sentence with k prepositional phrases.

Evidence Expected

Pseudocode, detailed traces and a correct analysis of the weaknesses of top-down parsing.

TASK 7 – Earley Parsing 5 Marks

Guiding Enquiry

How does the Earley algorithm use dynamic programming to avoid the failures of simple top-down parsing, and what does its chart contain?

Investigate and interpret:

- Define an Earley state (dotted rule with start position) and explain the three operations:

PREDICTOR, SCANNER and COMPLETER.

- Hand-simulate the algorithm on a helpdesk sentence of at least six words; show every chart entry (S[0]…S[n]) with its state, origin and the operation that created it.
- Recover the parse tree(s) from the chart by adding back-pointers and show how multiple trees are packed for an ambiguous sentence.
- Show how the algorithm handles a left-recursive rule and an ε-free ambiguous attachment without looping.
- Derive the complexity ($O(n^3)$ time, $O(n^2)$ space for the chart) and compare it with the top-down parser of Task 6 on the same sentence in terms of number of states generated.

Evidence Expected

A fully traced Earley chart, back-pointer tree recovery, and a justified complexity comparison.

TASK 8 – Feature Structures and Unification 4 Marks

Guiding Enquiry

How can attribute–value structures and unification replace multiplied categories to handle agreement and subcategorization elegantly?

Investigate and interpret:

- Define feature structures (attribute–value matrices), feature paths, reentrancy (structure sharing) and represent at least five lexical entries with features such as NUM, PER, CASE and SUBCAT.
- Define unification and subsumption, and work through at least three unifications, including one that fails.
- Augment at least six grammar rules with feature constraints (e.g., S → NP VP with 〈 NP AGREEMENT〉 = 〈 VP AGREEMENT〉) and show the unified feature structure for one complete parse.
- Redo Task 4 (agreement) and Task 5 (subcategorization) using features and compare the grammar size with the earlier non-feature version.
- Describe how unification can be integrated with the Earley parser (unification during the COMPLETER step) and what is lost in complexity.

Evidence Expected

Correct feature structures and unifications, a feature-based grammar fragment, and a comparison with the non-feature grammar.

TASK 9 – Probabilistic Context-Free Grammars 5 Marks

Guiding Enquiry

When a query has several valid parses, how can probabilities select the most plausible one?

Investigate and interpret:

- Define a PCFG: each rule $A \rightarrow \beta$ has a probability $P(A \rightarrow \beta \mid A)$, with the probabilities for each left-hand side summing to 1. Show that your grammar satisfies this condition.
- Estimate rule probabilities by maximum likelihood from your 20-sentence mini treebank, giving the counts, the formula and the resulting table; explain how you handle unseen rules.
- Compute the probability of at least two parse trees for an ambiguous sentence (e.g., PP-attachment) and choose the best parse; compute the probability of the sentence as the sum over its trees.
- Describe the probabilistic CKY (Viterbi) algorithm in pseudocode after conversion to CNF, and compute the table for a short sentence.
- Discuss the independence assumptions of PCFGs (context-freeness, position invariance), the preference for shorter trees, and the lack of lexical sensitivity, illustrating each with a failure case.

Evidence Expected

A well-formed PCFG with estimated probabilities, worked tree probability calculations, a Viterbi trace and a reasoned discussion of limitations.

TASK 10 – Implement and Test the Parsers 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python (plain code or NLTK) the grammar, a top-down parser, an Earley parser and a Viterbi/PCFG parser, with feature checks for agreement and subcategorization.
- Run all parsers on the 30-sentence test set and report, for each: parse found or not, number of parses, number of chart states/steps and run-time. • Include test cases for left recursion, ambiguous attachment, ungrammatical sentences and long sentences (up to 20 words), and a reference check against your hand-computed results from Tasks 7 and 9.
- Present the results in comparable tables and at least one plot (e.g., number of states against sentence length).

Evidence Expected

Working, documented code, a reproducible test procedure and results tables/graphs.

TASK 11 – Compare, Recommend and Defend the Parsing Design 2 Marks

Guiding Enquiry

Which grammar-and-parser combination would you deploy in the helpdesk, and what evidence supports your choice?

Investigate and interpret:

- Compare the top-down, Earley and probabilistic parsers on coverage, ambiguity handling, cost and ease of grammar maintenance, using your analysis and measured results.
- Recommend a design (e.g., feature-based CFG + Earley + PCFG ranking), stating its expected accuracy and limitations.
- Discuss manual effort (grammar writing and annotation) and the point at which a statistical approach becomes more economical than hand-written rules.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the fastest option alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The helpdesk must now accept conversational, multi-clause and partly ungrammatical student messages (for example, “Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?”). Messages may contain coordinated clauses, embedded questions, relative clauses and missing punctuation. At least 20% of the incoming messages are expected to contain a construction not covered by your grammar.

Re-evaluate your original design under the changed conditions. You may: **RETAIN**

→ MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the constructions in the new messages that your grammar cannot parse (embedded questions, multi-clause coordination, ellipsis, fragments).
- Identify the affected assumptions: controlled vocabulary, fully grammatical input, small ambiguity, hand-written coverage.
- Estimate how the longer sentences affect the number of parses and the parsing time.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended parser behave on the new input?

- Run or hand-trace at least three new messages through your original grammar and parsers and record what fails first.
- Estimate the growth of ambiguity and chart size with sentence length using your Task 7 and Task 10 results.
- Identify whether another parsing strategy handles the new input more economically.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the grammar and parser be extended, replaced, or supplemented?

- Choose retain, modify or redesign (e.g., add rules for embedded clauses, a robust/partial-parsing fallback, feature-based gap threading, a larger PCFG estimated from more data).
- Describe the revised grammar and parser and what changes in rules, features and probabilities.
- Determine the effect on coverage, ambiguity, parsing cost and annotation effort. **Evidence**

Expected

A design response with clear linguistic and algorithmic reasoning and quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one given the evidence you generated?

- Use your derivations, traces, simulation results, coverage figures and effort estimates.
- Show that the decision follows logically from the evidence and state its remaining limitations.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Design a Context-Free Grammar for English Syntax	5
Task 2 – Context-Free Rules, Derivations and Parse Trees	3
Task 3 – Sentence-Level Constructions	4
Task 4 – Agreement	3
Task 5 – Subcategorization	4
Task 6 – Top-Down Parsing	4
Task 7 – Earley Parsing	5
Task 8 – Feature Structures and Unification	4
Task 9 – Probabilistic Context-Free Grammars	5
Task 10 – Implement and Test the Parsers	3
Task 11 – Compare, Recommend and Defend the Parsing Design	2

Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

INTRODUCTION

Natural Language Processing (NLP) enables computers to understand and process human language. It is used in chatbots, virtual assistants, search engines and automated helpdesk systems.

A university helpdesk receives questions related to examinations, timetables, hostel facilities, library services, fees and placements. These questions may be written in different sentence structures and may contain grammatical ambiguity.

For example, the sentence:

"Show the students with the late fee notice."

may have two interpretations. The students may have the notice, or the notice may be used to identify the students.

To understand these sentences, the system requires a grammatical structure. This assignment uses Context-Free Grammars (CFG), Parse Trees, Top-Down Parsing, Earley Parsing, Feature Structures and Probabilistic Context-Free Grammars.

The main objective is to develop a grammar-based parsing system that identifies sentence structures and selects the most appropriate interpretation.

TASK 1 – DESIGN A CONTEXT-FREE GRAMMAR FOR ENGLISH SYNTAX

1.1 Definition of Context-Free Grammar

A Context-Free Grammar (CFG) is a formal system used to describe the grammatical structure of sentences.

A CFG is represented as:

[
G=(N,\Sigma,R,S)
]

Where:

- N = Set of non-terminal symbols.
- Σ = Set of terminal symbols.
- R = Set of production rules.
- S = Start symbol.

Non-terminals represent grammatical categories such as Sentence, Noun Phrase and Verb Phrase. Terminals represent actual words.

1.2 Non-Terminals and Terminals

Symbol	**Meaning**
S	Sentence
NP	Noun Phrase
VP	Verb Phrase
PP	Prepositional Phrase
AdjP	Adjective Phrase
AdvP	Adverb Phrase
Det	Determiner
N	Noun
V	Verb
Aux	Auxiliary Verb
Pron	Pronoun
Prep	Preposition

Conj	Conjunction
CP	Complement Phrase
RelClause	Relative Clause

1.3 Context-Free Grammar Rules

The following grammar contains 40 phrase-structure rules.

Sentence Rules

1. S → NP VP
2. S → VP
3. S → Aux NP VP
4. S → WH NP VP
5. S → NP Aux VP
6. S → S Conj S
7. S → CP

Noun Phrase Rules

8. NP → Det Nom
9. NP → Pron
10. NP → ProperN
11. NP → Nom
12. NP → NP PP
13. NP → NP RelClause
14. NP → Det AdjP Nom
15. NP → Num Nom

Nominal Rules

16. Nom → N
17. Nom → Adj N
18. Nom → Nom PP
19. Nom → Nom Conj Nom

Verb Phrase Rules

20. VP → V
21. VP → V NP
22. VP → V NP NP
23. VP → V PP
24. VP → V NP PP
25. VP → V CP
26. VP → Aux VP
27. VP → VP PP
28. VP → Adv VP
29. VP → VP Conj VP

Prepositional Phrase Rules

30. PP → Prep NP
31. PP → Prep Pron

Adjective and Adverb Rules

32. AdjP → Adj
33. AdjP → Adv Adj
34. AdvP → Adv
35. AdvP → AdvP Adv

Complement and Relative Clause Rules

36. CP → Comp S
37. RelClause → RelPron VP
38. RelClause → RelPron Aux VP
39. RelClause → RelPron NP VP
40. NP → NP Conj NP

These rules cover common university helpdesk sentence structures. Some rules are simplified and can be extended with agreement and feature constraints.

1.4 Lexicon

The grammar is separated from its lexicon. The lexicon contains the actual words and their grammatical categories.

Category	Words
Nouns	student, teacher, exam, timetable, hostel, room, library, book, fee, receipt, course, result, notice, form, assignment
Proper Nouns	Chennai, Vellore, Archana, Anna University, SIMATS
Verbs	submit, check, show, tell, book, apply, attend, pay, issue, open, close, study, register, cancel, postpone
Determiners	the, a, an, this, that, these, those
Pronouns	I, you, he, she, they, we, me, him, her, them
Prepositions	in, on, at, for, near, with, from, to
Adjectives	late, available, pending, new, old, important, final
Adverbs	quickly, recently, tomorrow, yesterday, here
Auxiliary Verbs	is, are, was, were, do, does, did, can, will, should
Conjunctions	and, or, but
WH Words	what, when, where, why, who, which, how
Complementisers	that, whether, if
Relative Pronouns	who, which, that
Numbers	one, two, three, four, five

This provides a domain vocabulary of more than 60 words and expressions.

1.5 Constituency, Head and Phrasal Category

A constituent is a group of words that functions as a single grammatical unit.

A head is the central word of a phrase that determines its grammatical category.

Examples:

1. **The student submitted the form.**
   "The student" is an NP, and student is its head.
2. **The teacher checked the timetable.**
   "Checked the timetable" is a VP, and checked is its head.
3. **The room near the library is available.**
   "Near the library" is a PP, and near is its head.

Constituency Tests

- Substitution: "The student" can be replaced by "She".
- Movement: "Near the library" can be moved to another position in the sentence.
- Coordination: "The timetable and the examination notice" can be combined into one coordinated NP.

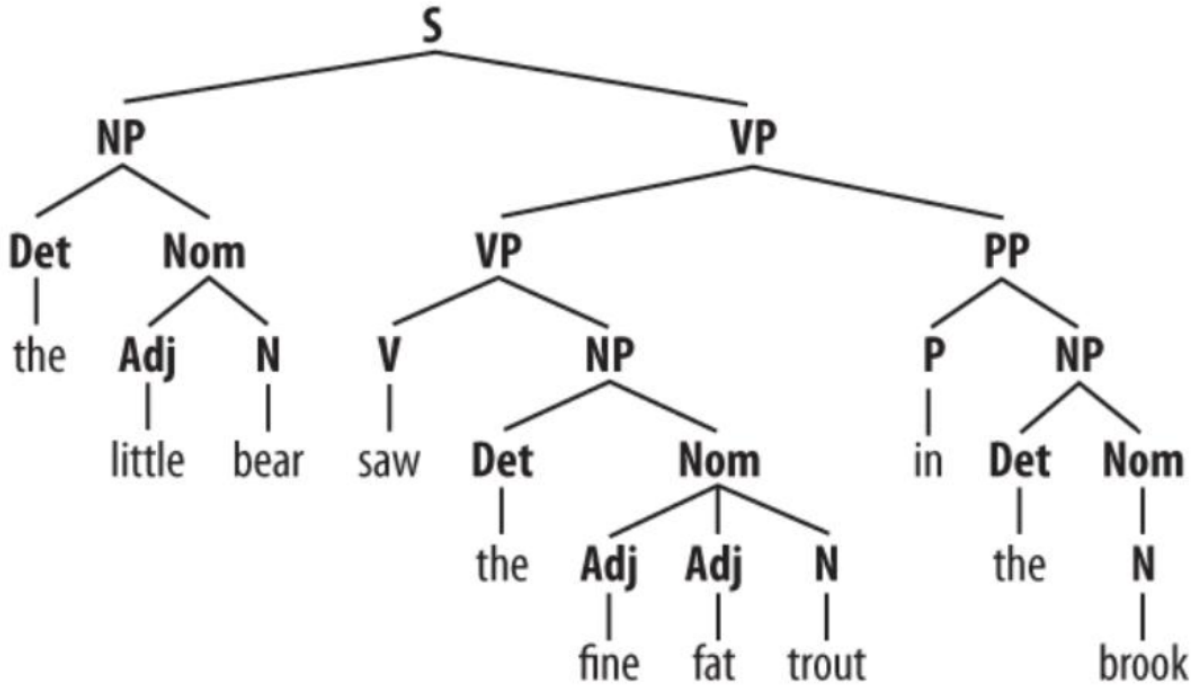


Figure 1 – Context-Free Grammar Parse Tree

1.6 Why CFG Is Context-Free

A CFG is context-free because the left-hand side of each production rule contains only one non-terminal symbol.

For example:

[
NP\rightarrow Det\ N
]

The rule can be applied independently of the surrounding symbols.

However, English agreement and certain cross-serial dependencies can be difficult to represent using a simple CFG without additional features or grammar extensions.

TASK 2 – CONTEXT-FREE RULES, DERIVATIONS AND PARSE TREES

2.1 Derivation

A derivation is the process of generating a sentence by repeatedly applying grammar production rules.

There are two main types:

- Leftmost derivation: Expands the leftmost non-terminal first.
- Rightmost derivation: Expands the rightmost non-terminal first.

2.2 Example of Leftmost Derivation

Sentence: "The student submitted the form."

[
S\Rightarrow NP\ VP
]

[
\Rightarrow Det\ Nom\ VP
]

[
\Rightarrow The\ N\ VP
]

[
\Rightarrow The\ student\ VP
]

[
\Rightarrow The\ student\ V\ NP
]

[
\Rightarrow The\ student\ submitted\ Det\ N
]

[
\Rightarrow The\ student\ submitted\ the\ form
]

2.3 Rightmost Derivation

[
S\Rightarrow NP\ VP
]

[
\Rightarrow NP\ V\ NP
]

[
\Rightarrow NP\ V\ Det\ N
]

[
\Rightarrow NP\ V\ the\ form
]

[
\Rightarrow NP\ submitted\ the\ form
]

[
\Rightarrow Det\ N\ submitted\ the\ form
]

[
\Rightarrow The\ student\ submitted\ the\ form
]

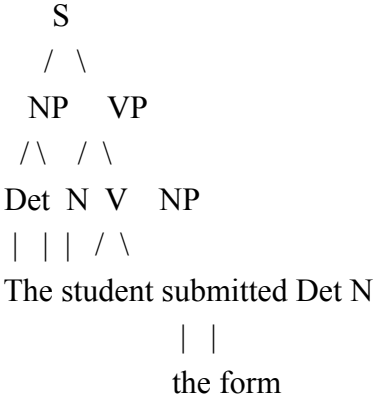
2.4 Parse Tree

A parse tree represents the grammatical structure of a sentence.

Bracketed Representation:

```
(S
  (NP (Det The) (N student))
  (VP
    (V submitted)
    (NP (Det the) (N form))))
```

Tree Structure:



The tree has:

- 11 nodes.
- Height of 4 edges from root to deepest leaf.
- 6 rule applications when lexical productions are included.

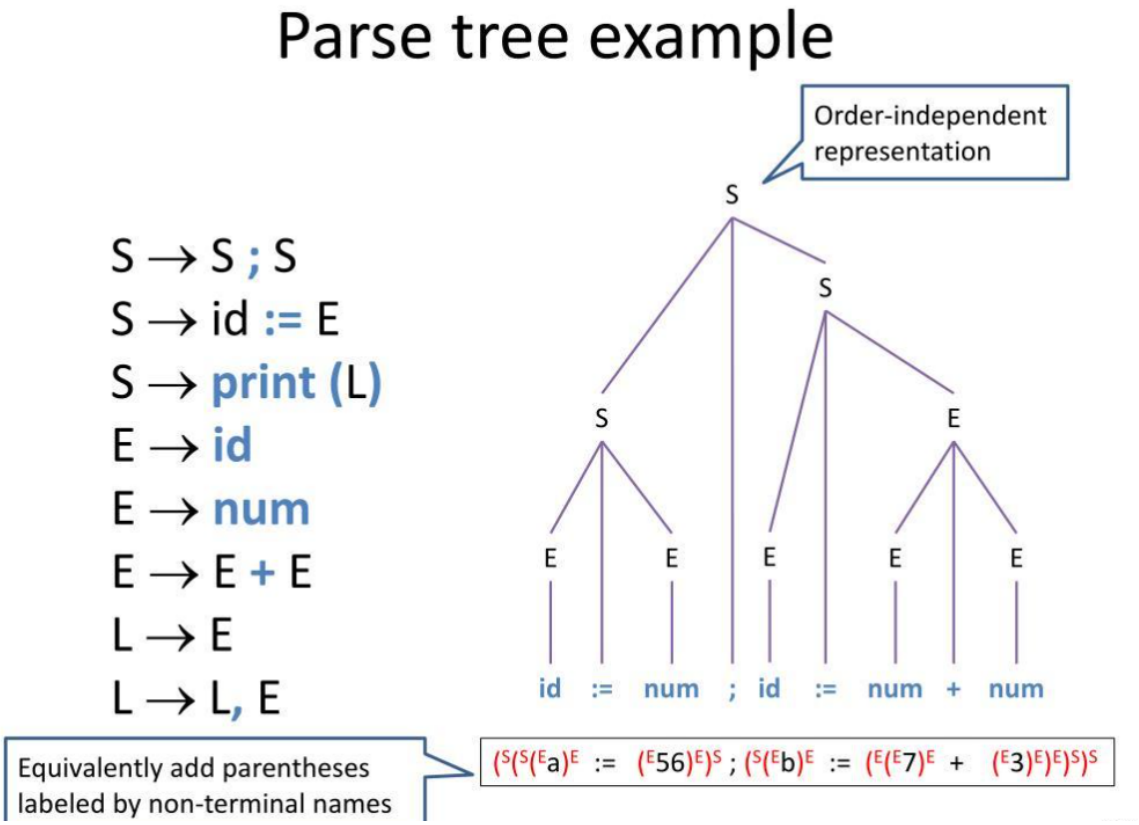


Figure 2 – Derivation and Parse Tree

2.5 Five Sample Sentences

No.	Sentence	Structure
1	The student submitted the form.	NP + VP
2	The teacher checked the timetable.	NP + VP
3	Submit the assignment.	Imperative VP

4	Is the library open?	Auxiliary + NP + VP
5	Where is the examination hall?	WH Question

Each sentence can be derived using the proposed grammar and represented as a parse tree.

2.6 Structural Ambiguity

Structural ambiguity occurs when one sentence has more than one possible parse tree.

Example:

"Book the room near the library for the seminar."

Interpretation 1: The room is near the library.

VP
├── V Book
├── NP
│ ├── Det the
│ ├── N room
│ └── PP near the library
└── PP for the seminar

Interpretation 2: The booking should be made near the library for the seminar.

VP
├── V Book
├── NP the room
└── PP
 ├── near the library
 └── for the seminar

The difference is the attachment of the prepositional phrase.

Another example is:

"Show the students with the late fee notice."

The PP may describe the students or indicate which notice should be shown.

2.7 Sentence Membership

To determine whether a sentence belongs to L(G):

1. Start with the start symbol S.
2. Apply valid grammar production rules.
3. Match terminal words with the input.
4. Continue until all input words are consumed.
5. Accept if a complete derivation is obtained.

A sentence that cannot be generated by the grammar is rejected.

The main limitation is that an incomplete grammar may reject grammatically correct English sentences.

TASK 3 – SENTENCE-LEVEL CONSTRUCTIONS

3.1 Types of Sentences

The university helpdesk must understand four major sentence constructions.

1. Declarative Sentences

Declarative sentences provide information.

Example:

"The student submitted the assignment."

Rule:

[
S\rightarrow NP\ VP
]

2. Imperative Sentences

Imperative sentences express commands or requests.

Example:

"Check the examination timetable."

Rule:

[
S\rightarrow VP
]

The subject "you" is generally understood.

3. Yes/No Questions

These questions expect a yes or no answer.

Example:

"Is the library open?"

Rule:

[
S\rightarrow Aux\ NP\ VP
]

4. WH-Questions

WH-questions begin with words such as what, when, where, why and how.

Example:

"When will the exams begin?"

Rule:

[
S\rightarrow WH\ Aux\ NP\ VP
]

Subject WH-question:

"Who submitted the form?"

[
S\rightarrow WH\ VP
]

3.2 Auxiliary Inversion

In yes/no questions, the auxiliary verb moves before the subject.

Declarative:

"The library is open."

Question:

"Is the library open?"

This process is called subject-auxiliary inversion.

3.3 WH-Movement

WH-movement occurs when a question word appears at the beginning of a sentence.

Example:

"The examination begins when."

Becomes:

"When does the examination begin?"

The grammar can represent the original position using a gap:

(S
  (WH When)
  (Aux does)
  (NP the examination)
  (VP begin t))

Here, t represents the original position of the WH expression.

3.4 Subordinate and Relative Clauses

A subordinate clause depends on another clause.

Example:

"The exams that were postponed will be held tomorrow."

The phrase "that were postponed" is a relative clause describing the exams.

3.5 Coordination

Coordination joins two or more grammatical units.

Example:

"The student paid the fees and hostel charges."

The word "and" connects two noun phrases.

3.6 Effect on Grammar Size

Adding questions, relative clauses, coordination and subordinate clauses increases the number of production rules and possible parse structures.

It also increases ambiguity because one phrase may attach to different parts of a sentence.

A plain CFG becomes difficult to maintain when long-distance dependencies and complex embedded clauses are introduced.

TASK 4 – AGREEMENT

4.1 Definition

Agreement is the grammatical relationship between words in a sentence.

The main agreement types are:

1. Subject–verb agreement.
2. Determiner–noun agreement.
3. Pronoun case agreement.

4.2 Subject–Verb Agreement

A singular subject generally requires a singular verb, while a plural subject requires a plural verb.

Examples:

- The student submits the form.
- The students submit the forms.

Incorrect:

- The students submits the form.

4.3 Determiner–Noun Agreement

Determiners must match the number of nouns.

Examples:

- This student.
- These students.
- That book.
- Those books.

4.4 Pronoun Case

Pronouns change according to their grammatical role.

Subject Case	**Object Case**
I	Me
He	Him
She	Her
We	Us

They	Them

Example:

"She submitted the form."

" The teacher called her."

4.5 Rule Multiplication

Agreement can initially be implemented by creating separate non-terminals.

[
S\rightarrow NP_{sg}\ VP_{sg}
]

[
S\rightarrow NP_{pl}\ VP_{pl}
]

This prevents singular subjects from being combined with plural verbs.

However, combining person, number, case and gender creates many possible combinations.

For example:

2 numbers × 3 persons × 2 cases × 3 gender values = 36 feature combinations.

As the number of grammatical features increases, rule multiplication becomes difficult to maintain.

4.6 Feature-Based Alternative

Instead of creating many separate rules, grammatical features can be attached to words.

Example:

student: [NUM = singular]

students: [NUM = plural]

The verb must have matching number features.

This is more economical and is explained further in Task 8.

4.7 Agreement Testing

Sentence	**Expected Result**
The student submits the form.	Accept
The students submit the form.	Accept
The teacher checks the result.	Accept
The teachers check the results.	Accept
She pays the fee.	Accept
They are attending the exam.	Accept
The students submits the form.	Reject
The student submit the form.	Reject
These student is present.	Reject
This students are present.	Reject
Me submitted the form.	Reject
Him is attending the class.	Reject

The agreement-aware grammar rejects the ungrammatical sentences while accepting the correct ones.

TASK 5 – SUBCATEGORIZATION

5.1 Definition

Subcategorization describes the types of complements that a verb requires.

Different verbs require different sentence structures.

For example:

- Submit requires an object.
- Sleep does not require an object.
- Send requires an object and a recipient.
- Tell can take a sentence complement.

5.2 Five Subcategorization Frames

Frame	**Structure**	**Example**
Intransitive	V	The portal crashed.
Transitive	V + NP	Submit the form.
Ditransitive	V + NP + NP	Send me the receipt.
Sentential Complement	V + CP	Tell me that the fee is paid.
PP Complement	V + PP	Apply for a hostel room.

5.3 Grammar Rules

[
VP\rightarrow V_{intr}
]

[
VP\rightarrow V_{trans}\ NP
]

[
VP\rightarrow V_{ditrans}\ NP\ NP
]

[
VP\rightarrow V_{comp}\ CP
]

[
VP\rightarrow V_{pp}\ PP
]

5.4 Subcategorization Lexicon

Verb	**Subcategorization**
Sleep	Intransitive
Arrive	Intransitive
Submit	Transitive
Check	Transitive
Send	Ditransitive
Give	Ditransitive
Tell	Sentential complement
Know	Sentential complement

Apply	PP complement
Depend	PP complement

5.5 Complements and Adjuncts

A complement is required or selected by a particular verb, while an adjunct adds optional information.

Example:

"Apply for a hostel room."

The PP "for a hostel room" is selected by the verb apply.

"Submit the form yesterday."

The word "yesterday" is an adjunct because it provides optional time information.

5.6 Accept and Reject Examples

Sentence	**Result**	**Reason**
The portal crashed.	Accept	Intransitive
Submit the form.	Accept	Transitive
Send me the receipt.	Accept	Ditransitive
Tell me that the fee is paid.	Accept	Sentential complement
Apply for a hostel room.	Accept	PP complement
Submit.	Reject in this frame	Required object missing
The portal crashed the form.	Reject	Intransitive verb used incorrectly

Send the receipt me.	Reject in this frame	Incorrect complement order

A verb may support multiple frames. For example, "book" can be used transitively or with additional PP information. This increases ambiguity and parsing effort.

TASK 6 – TOP-DOWN PARSING

6.1 Definition

Top-Down Parsing is a parsing method that begins with the start symbol S and attempts to generate the input sentence using grammar rules.

It is also called goal-driven parsing.

6.2 Working Procedure

1. Start from S.
2. Select a production rule.
3. Expand the leftmost non-terminal.
4. Match terminals with input words.
5. If matching fails, backtrack.
6. Continue until the complete sentence is recognised.

6.3 Pseudocode

```
TOP_DOWN_PARSE(sentence):
   start with S

   if all words are matched:
      return SUCCESS

   choose a grammar rule
   expand the non-terminal

   if terminal matches:
      continue parsing
```

otherwise:
backtrack and try another rule

return FAILURE if no rule works

6.4 Example Trace

Sentence:

"The student submitted the form."

Step	Current Operation
1	Start with S
2	Apply S → NP VP
3	Expand NP → Det N
4	Match "The"
5	Match "student"
6	Expand VP → V NP
7	Match "submitted"
8	Expand NP → Det N
9	Match "the form"
10	Accept sentence

6.5 Ambiguous Sentence

Sentence:

"Book the room near the library."

The parser may initially attach the PP "near the library" to the NP "the room" or to the VP "Book the room".

If one interpretation fails to match the remaining input, the parser backtracks and tries another rule.

6.6 Left Recursion Problem

Consider:

[
NP\rightarrow NP\ PP
]

A top-down parser may repeatedly expand NP into NP PP without consuming any input.

This causes infinite recursion.

A left-recursion-free form is:

[
NP\rightarrow Nom\ NP'
]

[
NP'\rightarrow PP\ NP'|\epsilon
]

This allows the parser to process additional PPs without repeatedly expanding the same leftmost non-terminal.

6.7 Time Complexity

In the worst case, a simple top-down parser may explore an exponential number of derivations because it repeatedly tries alternative rules and backtracks.

Repeated subtrees may also be parsed multiple times.

Therefore, top-down parsing is suitable for simple grammars but becomes inefficient for ambiguous and recursive sentences.

TASK 7 – EARLEY PARSING

7.1 Definition

Earley Parsing is a chart-parsing algorithm used to recognise sentences using a CFG.

It is useful because it stores intermediate parsing results and avoids repeatedly processing the same states.

An Earley state contains:

- Grammar production rule.
- Dot showing how much of the rule has been recognised.
- Origin position in the input.

Example:

[
[S\rightarrow NP\bullet VP,\ 0]
]

The dot indicates that NP has been recognised and VP is expected next.

7.2 Three Main Operations

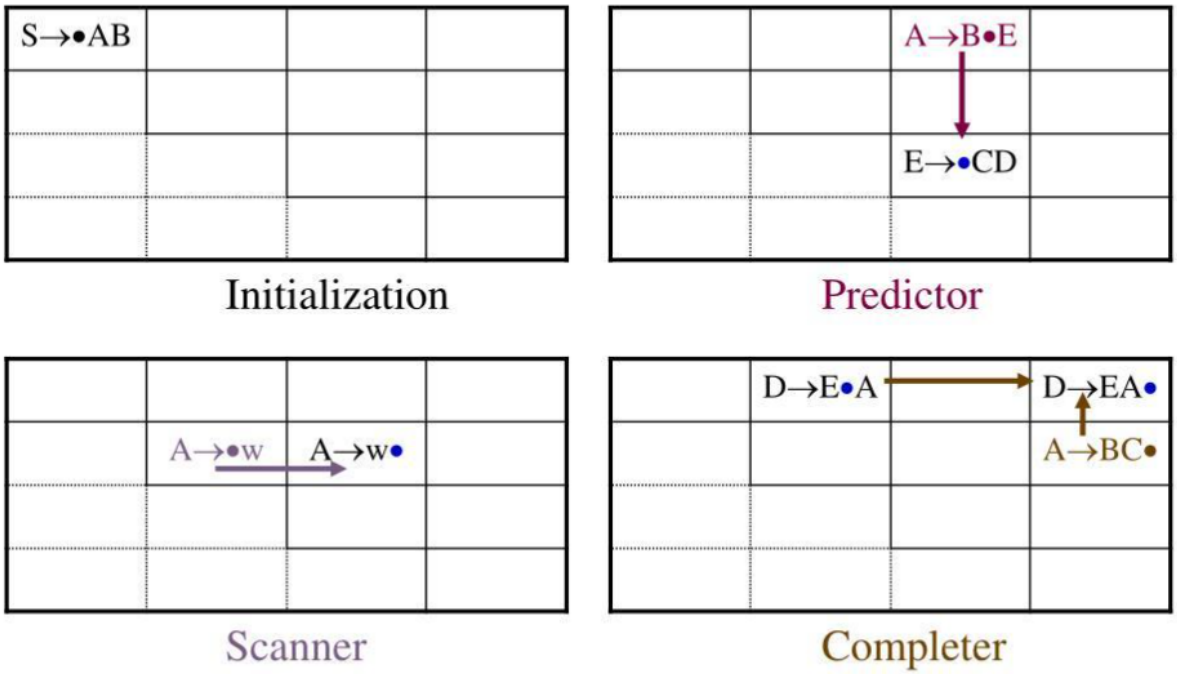


Figure 3 – Earley Parsing Chart

1. Predictor

Predictor adds possible grammar rules when the next expected symbol is a non-terminal.

2. Scanner

Scanner matches the expected terminal with the current input word.

3. Completer

Completer advances previously waiting states when a constituent has been completed.

7.3 Earley Parsing Example

Sentence:

"The student submitted the form."

Tokens:

0 The, 1 student, 2 submitted, 3 the, 4 form.

A simplified chart trace is shown below. The complete state notation is:

[Production, Origin]

Chart	**State / Operation**
S0	S' → • S [Predictor]
S0	S → • NP VP [Predictor]
S0	NP → • Det N [Predictor]
S0	Det → • the [Predictor]
S1	Det → the • [Scanner]
S1	NP → Det • N [Completer]
S1	N → • student [Predictor]
S2	N → student • [Scanner]
S2	NP → Det N • [Completer]
S2	S → NP • VP [Completer]
S2	VP → • V NP [Predictor]
S2	V → • submitted [Predictor]
S3	V → submitted • [Scanner]

S3	VP → V • NP [Completer]
S3	NP → • Det N [Predictor]
S4	Det → the • [Scanner]
S4	NP → Det • N [Completer]
S5	N → form • [Scanner]
S5	NP → Det N • [Completer]
S5	VP → V NP • [Completer]
S5	S → NP VP • [Completer]

The completed start state indicates that the sentence is successfully parsed.

7.4 Handling Ambiguity

Earley parsing can preserve multiple possible parse structures for the same sentence.

For example:

"Book the room near the library."

The chart can contain separate structures for PP attachment to NP and VP.

These alternatives can be represented using back-pointers or a shared packed parse forest.

7.5 Left Recursion

Earley parsing handles left-recursive rules because it stores states and avoids adding duplicate states repeatedly.

Unlike simple recursive-descent parsing, it does not become trapped in infinite left-recursive expansion.

7.6 Complexity

For a sentence of length n:

- Worst-case time complexity: $O(n^3)$.
- Space complexity: $O(n^2)$.

For unambiguous grammars, parsing can be more efficient.

Earley parsing is more suitable than basic top-down parsing for recursive and ambiguous helpdesk sentences.

TASK 8 – FEATURE STRUCTURES AND UNIFICATION

8.1 Definition

Feature Structures represent grammatical information using attribute-value pairs.

They help describe agreement, number, person, case and verb complement requirements.

Example:

[WORD: student
NUM: singular
PER: third
CASE: nominative]

8.2 Feature Paths

A feature path identifies a particular value inside a feature structure.

For example:

NP.NUM

refers to the number feature of a noun phrase.

8.3 Reentrancy

Reentrancy occurs when two different parts of a feature structure share the same value.

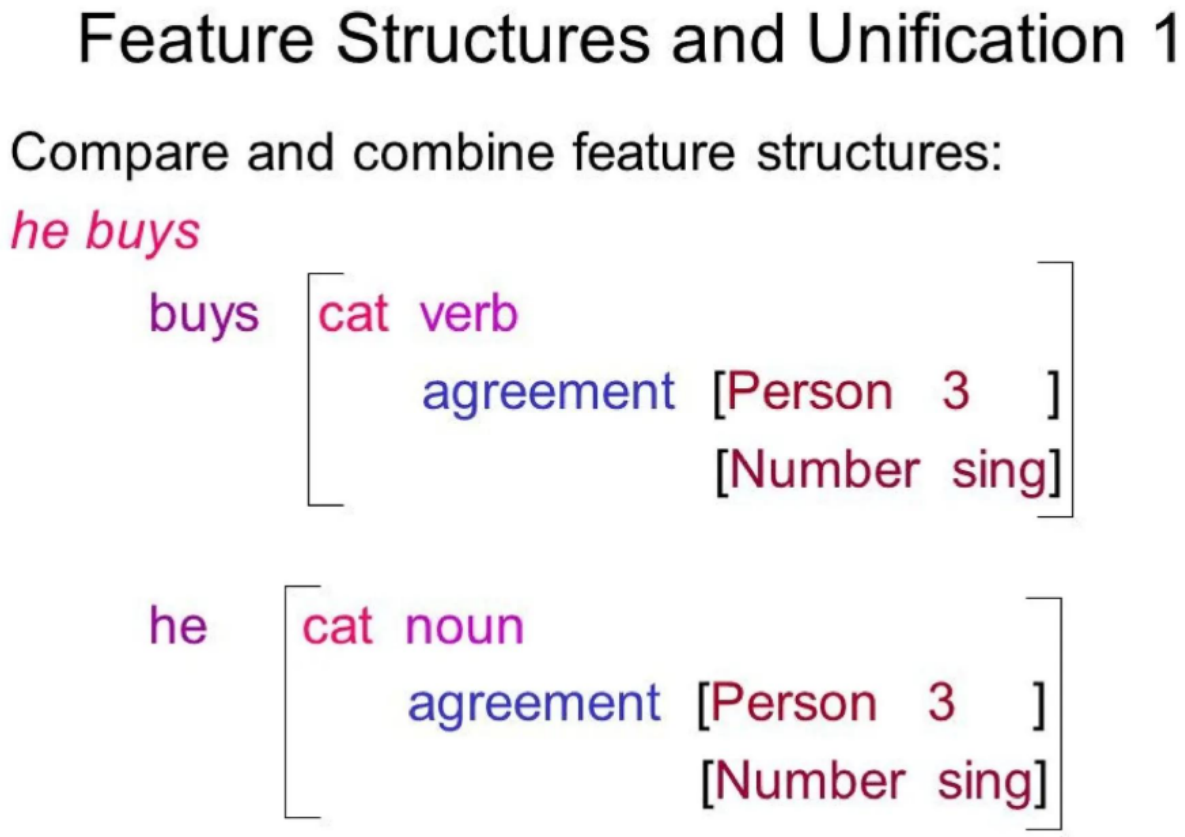
For example, the subject NP and the VP can share the same agreement feature.

This ensures that subject and verb agreement is maintained.

8.4 Lexical Feature Structures

Word	**Features**
Student	NUM=SG, CASE=NOM
Students	NUM=PL, CASE=NOM
Submits	NUM=SG, PER=3
Submit	NUM=PL, PER=3
Me	CASE=OBJ

Figure 4 – Feature Structures and Unification



8.5 Unification

Unification combines two compatible feature structures into one.

Example 1:

NP: [NUM = SG]
VP: [NUM = SG]

Result: Successful unification.

Example 2:

NP: [NUM = PL]
VP: [NUM = PL]

Result: Successful unification.

Example 3:

NP: [NUM = PL]
VP: [NUM = SG]

Result: Failure because the number features conflict.

8.6 Feature-Based Grammar Rules

1. S → NP VP, where NP.NUM = VP.NUM.
2. NP → Det N, where Det.NUM = N.NUM.
3. VP → V NP, where V.SUBCAT = transitive.
4. VP → V NP NP, where V.SUBCAT = ditransitive.
5. VP → V CP, where V.SUBCAT = sentential.
6. VP → V PP, where V.SUBCAT = PP complement.

These rules enforce grammatical agreement and verb complement requirements.

8.7 Complete Feature-Based Example

Sentence:

"The student submits the form."

Subject NP:

[NUM = SG
 PER = 3
 CASE = NOM]

Verb:

[NUM = SG
 PER = 3
 SUBCAT = TRANS]

Object NP:

[NUM = SG

CASE = OBJ]

The subject and verb features unify successfully, so the sentence is accepted.

8.8 Comparison with Ordinary CFG

Ordinary CFG	**Feature-Based CFG**
Uses separate grammar categories	Uses attribute-value pairs
Agreement requires extra rules	Agreement uses unification
Grammar may become large	More compact grammar
Difficult to maintain	Easier to extend

8.9 Unification in Earley Parsing

Feature unification can be performed during the Completer operation of Earley parsing.

When a constituent is completed, its features are checked against the waiting grammar state.

If the features are compatible, the parser advances. Otherwise, that parse path is rejected.

Feature checking adds computational cost because the parser must maintain and compare feature structures.

TASK 9 – PROBABILISTIC CONTEXT-FREE GRAMMARS

9.1 Definition

A Probabilistic Context-Free Grammar (PCFG) is a CFG in which every production rule is assigned a probability.

It helps select the most likely parse tree when a sentence has multiple grammatical interpretations.

A PCFG is represented as:

$$
G=(N,\Sigma,R,S,P)
$$

Where P represents the probability assigned to grammar rules.

For every non-terminal A:

$$
\sum_{\beta}P(A\rightarrow\beta)=1
$$

9.2 Maximum Likelihood Estimation

Rule probabilities are calculated using the frequency of each rule in an annotated treebank.

$$
P(A\rightarrow\beta)=
\frac{Count(A\rightarrow\beta)}
{Count(A)}
$$

Example:

Suppose VP occurs 20 times:

- VP → V NP occurs 12 times.
- VP → V occurs 5 times.
- VP → VP PP occurs 3 times.

Therefore:

$$
P(VP\rightarrow V\ NP)=\frac{12}{20}=0.6
$$

$$
P(VP\rightarrow V)=\frac{5}{20}=0.25
$$

[
P(VP\rightarrow VP\ PP)=\frac{3}{20}=0.15
]

The total is:

[
0.6+0.25+0.15=1
]

9.3 Example PCFG Rule Probabilities

The following is an illustrative rule-count table.

Grammar Rule	Count	Probability
S → NP VP	20	1.00
NP → Det N	12	0.60
NP → Pron	5	0.25
NP → ProperN	3	0.15
VP → V NP	12	0.60
VP → V	5	0.25
VP → VP PP	3	0.15
PP → Prep NP	20	1.00

Each set of rules with the same left-hand side sums to one.

9.4 Parse Tree Probability

The probability of a parse tree is calculated by multiplying the probabilities of all rules used in that tree.

[
P(T)=\prod_i P(r_i)
]

For example:

"Book the room near the library."

The PP may attach to the noun phrase or verb phrase.

Suppose the probabilities are:

- NP attachment = 0.30
- VP attachment = 0.70

The VP attachment is selected because it has the higher probability.

The probability of the sentence is the sum of the probabilities of all valid parse trees:[
P(S)=\sum_{T\in Parse(S)}P(T)
]

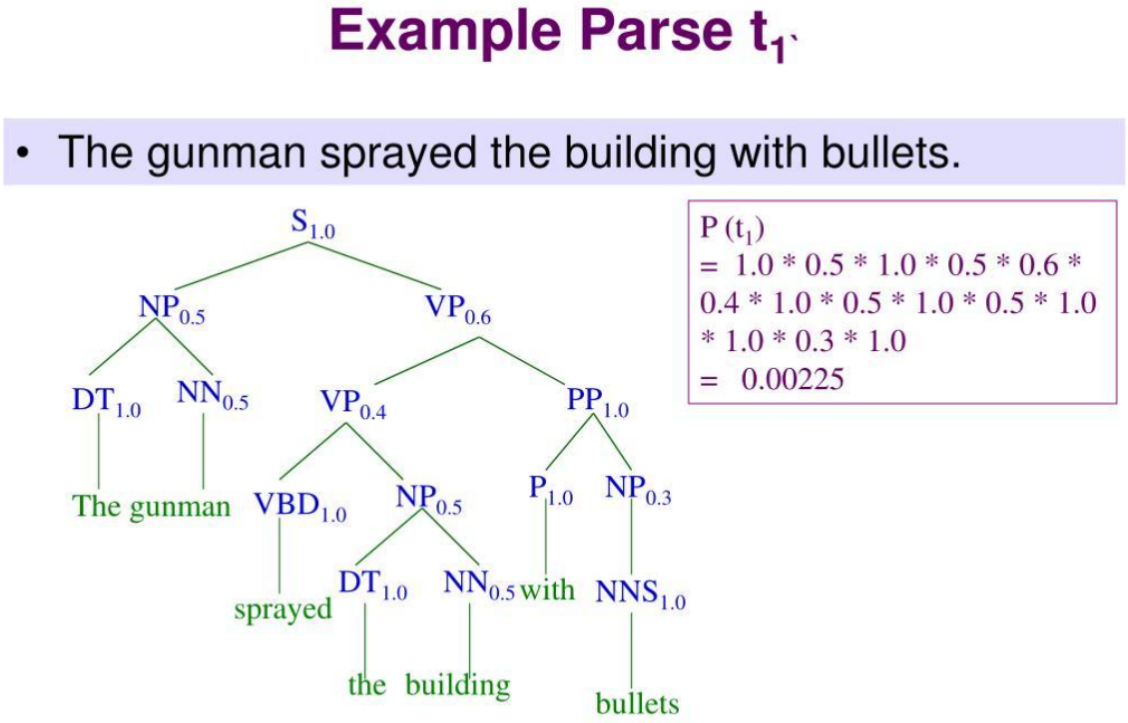


Figure 5 – Probabilistic CFG Parse Tree

9.5 Probabilistic CKY Algorithm

The Cocke–Kasami–Younger (CKY) algorithm finds the most probable parse using dynamic programming.

It generally requires a grammar in Chomsky Normal Form (CNF).

CNF rules have the forms:

[
A\rightarrow BC
]

or

[
A\rightarrow a
]

Conversion Example:

Original:

[
VP\rightarrow V\ NP\ PP
]

Binarised form:

[
VP\rightarrow V\ X
]

[
X\rightarrow NP\ PP
]

The Viterbi CKY algorithm stores the best probability for each constituent and selects the highest-probability complete parse.

9.6 PCFG Limitations

1. Context independence: Rule probabilities depend only on the parent non-terminal.
2. Position independence: The same rule probability is used in different sentence contexts.
3. Lexical sensitivity: Basic PCFGs may not consider the meaning or identity of particular words.
4. Tree preference: Probabilities may favour structurally common trees even when the intended meaning is different.

PCFGs are useful for ranking parses, but they may require lexical or contextual information for improved accuracy.

TASK 10 – IMPLEMENT AND TEST THE PARSERS

10.1 Implementation

The system can be implemented using Python and NLTK.

The main modules include:

1. Grammar definition.
2. Lexicon construction.
3. Top-down parsing.
4. Earley parsing.
5. Feature-based agreement checking.
6. PCFG probability calculation.
7. Viterbi parse selection.
8. Testing and performance evaluation.

10.2 Python Implementation

```
import nltk
from nltk import CFG
from nltk.parse import RecursiveDescentParser
from nltk.parse import EarleyChartParser

grammar = CFG.fromstring("""
S -> NP VP

NP -> Det N
NP -> Pron
NP -> ProperN

VP -> V NP
VP -> V
VP -> VP PP

PP -> Prep NP

Det -> 'the' | 'a'
N -> 'student' | 'form' | 'room'
Pron -> 'she'
ProperN -> 'Chennai'

V -> 'submitted' | 'checked' | 'book'

Prep -> 'near' | 'in'
""")
```

```
sentence = [
    'the', 'student', 'submitted',
    'the', 'form'
]

# Top-down parsing
top_parser = RecursiveDescentParser(grammar)
print("Top-down parsing:")
for tree in top_parser.parse(sentence):
    print(tree)

# Earley chart parsing
earley_parser = EarleyChartParser(grammar)
print("Earley parsing:")
for tree in earley_parser.parse(sentence):
    print(tree)
```

10.3 Test Cases

No.	Input Sentence	Expected Result
1	The student submitted the form.	Accept
2	The teacher checked the timetable.	Accept
3	Submit the assignment.	Accept
4	Is the library open?	Accept
5	Where is the examination hall?	Accept
6	The students submits the form.	Reject with agreement constraints
7	The portal crashed the form.	Reject with subcategorization

8	Book the room near the library.	Multiple parses
9	The teacher is available.	Accept
10	Me submitted the form.	Reject with case constraints

The complete 30-sentence test set should include declaratives, imperatives, yes/no questions, WH-questions and at least five ambiguous sentences.

10.4 Performance Evaluation

Parser	**Main Measurement**	**Expected Behaviour**
Top-Down	Steps and runtime	May backtrack
Earley	Chart states and runtime	Reuses intermediate states
PCFG / Viterbi	Best parse and probability	Selects most probable tree

The implementation should record:

- Number of parses.
- Number of chart states.
- Runtime.
- Accepted and rejected sentences.
- Best parse probability.

10.5 Visualisation

The graph should compare the number of states generated by top-down and Earley parsing for sentences of increasing length.

The experiment helps demonstrate the effect of repeated parsing and chart-based state reuse.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE PARSING DESIGN

11.1 Parser Comparison

Feature	Top-Down	Earley	PCFG
Parsing Approach	Goal-driven	Chart-based	Probability-based
Ambiguity	Backtracking	Multiple parses	Ranks parses
Left Recursion	Problematic	Supported	Depends on parsing algorithm
Efficiency	Can be expensive	Avoids repeated states	Efficient with suitable algorithms
Main Purpose	Basic parsing	General CFG parsing	Best parse selection

11.2 Recommended Design

The recommended architecture is:

Feature-Based CFG + Earley Parser + PCFG Ranking

- Feature-based CFG checks agreement and subcategorization.
- Earley parsing generates valid sentence structures.
- PCFG ranks alternative parse trees.
- The highest-probability parse is selected.

This combination provides a balance between grammatical accuracy, ambiguity handling and computational efficiency.

11.3 Manual Effort and Cost

Manual work is required to create grammar rules, prepare the lexicon, annotate parse trees and test ambiguous sentences.

As the number of helpdesk queries increases, maintaining all grammar rules manually becomes difficult.

At that stage, a larger annotated treebank and probabilistic or learned parsing methods can reduce manual rule-writing effort.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

A. Impact Analysis

The original system was designed for controlled and relatively simple university helpdesk queries.The new requirement introduces conversational language, multiple clauses, embedded questions, relative clauses and incomplete sentences.

Example:

"Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?"

The new requirements introduce several challenges:

1. Embedded questions.
2. Multiple coordinated clauses.
3. Relative clauses.
4. Missing punctuation.
5. Incomplete or conversational expressions.
6. Increased structural ambiguity.

The original grammar may not cover all these constructions.

B. Evaluation of the Original Design

Consider the following three new messages.

Message	**New Challenge**	**Possible Failure**

Can you tell me when the exams will be held?	Embedded question	WH clause not covered
The students who paid the fees can collect receipts.	Relative clause	Missing relative-clause rules
Hi, is the hostel open and can I apply now?	Coordination and conversation	Multiple sentence structures

The original parser may reject these sentences or produce incomplete parse trees because the grammar was designed for a limited language fragment.

Longer sentences also increase the number of possible parsing states and alternative structures.

C. Adaptation or Redesign

Selected Approach: MODIFY

The existing grammar and parsing system will be retained and extended.

Proposed Changes

1. Add grammar rules for embedded questions.
2. Extend relative-clause rules.
3. Add coordinated sentence structures.
4. Introduce feature-based gap handling for WH dependencies.
5. Add partial parsing for incomplete messages.
6. Expand the lexicon with conversational expressions.
7. Re-estimate PCFG probabilities using a larger annotated treebank.
8. Retain Earley parsing for recursive and ambiguous constructions.

Revised Architecture

Conversational Input → Text Normalisation → Sentence Segmentation → Feature-Based CFG → Earley Parsing → PCFG Ranking → Best Parse Tree → Helpdesk Information Extraction

D. Evidence-Based Defence

Modification is more suitable than completely replacing the original grammar because CFG and Earley parsing remain useful for representing English sentence structures.

The extended grammar increases sentence coverage, while feature structures maintain agreement and subcategorization.Earley parsing handles recursive structures and ambiguity without repeatedly processing identical states. PCFG ranking selects the most probable interpretation.

The revised system should be tested using:

- Parse coverage.
- Number of valid parse trees.
- Earley chart states.
- Parsing runtime.
- PCFG ranking accuracy.

Remaining Limitations:

- Some conversational expressions may not follow standard grammar.
- Highly ambiguous sentences may produce several valid trees.
- A hand-written grammar cannot cover every possible English sentence.
- Basic PCFGs do not fully understand the meaning or context of a sentence.

Therefore, a modified feature-based CFG combined with Earley parsing and PCFG ranking is recommended for the university helpdesk.

CONCLUSION

This assignment presented a grammar-based NLP system for analysing university helpdesk queries.

Context-Free Grammars describe sentence structures, while derivations and parse trees represent how sentences are generated. Agreement and subcategorization improve grammatical correctness.Top-Down Parsing provides a simple method for exploring grammar rules, while Earley Parsing efficiently handles recursive and ambiguous sentence structures.

Feature Structures and Unification provide an economical way to enforce grammatical constraints. PCFGs assign probabilities to alternative parse trees and help select the most likely interpretation.

The Day-4 design challenge demonstrates the need to extend the grammar to support conversational language, embedded questions and multi-clause sentences.

Overall, a feature-based CFG combined with Earley parsing and probabilistic ranking provides a suitable foundation for a university helpdesk assistant.